

Ödev 2

Mini-Max Algoritması ve Alfa-Beta Budaması Teknikleri

Çin Damaları oyununda ajanı tasarlarken Mini-Max algoritması ve Alfa-Beta Budaması kullandım. Bu iki teknik, oyuncuların en iyi hamlesini seçmesini sağlar.

Mini-Max algoritması: Oyunun tüm hamle seçeneklerini değerlendirerek en iyi hamleyi bulmak amacıyla her oyuncunun dönüş sırasıyla hamle yapmasını sağlar. Ajanın her hamlesi, her oyuncunun en iyi hareketi seçmesini hedefler.

Alfa-Beta Budaması: Bu teknik, Mini-Max algoritmasını daha verimli hale getirir. Olası hesaplamaların bir kısmını atlayarak gereksiz hesaplamalardan kaçınır ve algoritmanın daha hızlı çalışmasını sağlar.

Ajanı, her oyuncu için sırayla en iyi hamleyi seçmek üzere programladım ve bu hamleler, taşların hedef bölgeye ulaşmasını sağlamak için optimize edilmiştir.

Durum, Hareket, Algı, Hedef Fonksiyonu ve Ardıl Fonksiyon Tanımlamaları

Durum (State) Fonksiyonu: Oyun tahtasının her bir hücrelerinde, oyuncunun taşları, rakip taşları veya boş alanlar bulunmaktadır. Oyun durumu, bu taşların yerlerini ve tahtadaki diğer bilgileri içerir.

Hareket (Move) Fonksiyonu: Hamle fonksiyonu, her oyuncunun taşlarını belirli bir mesafeye (maksimum 2 adım) hareket ettirmesine izin verir. Ayrıca, taşlar birbirlerinin üzerinden atlayarak da hareket edebilir.

Algı (Perception) Fonksiyonu: Algı fonksiyonu, ajanın tahtadaki mevcut durumunu algılamasına olanak tanır. Tahtadaki taşlar, boş alanlar ve rakip taşlar gözlemlenir.

Hedef (Goal) Fonksiyonu: Hedef fonksiyonu, oyuncunun hedef bölgesine ulaşmış olup olmadığını kontrol eder. Beyaz oyuncular alt sağ köşeye, siyah oyuncular ise üst sol köşeye ulaşmalıdır.

Ardıl (Successor) Fonksiyonu: Ardıl fonksiyonu, mevcut durumdan elde edilebilecek tüm geçerli hamleleri üretir. Bu fonksiyon, taşların atlama hareketlerini de içeren tüm olasılıkları değerlendirir.

Sezgi (Heuristic) Fonksiyonu: Bu fonksiyon, oyuncunun taşlarının hedef bölgesine ne kadar yakın olduğunu hesaplar. Beyaz oyuncu için puan, taşların yukarıya doğru hareket etmesiyle azalırken, siyah oyuncu için tam tersi bir şekilde, taşların aşağıya doğru hareket etmesiyle artar.

Mini-Max Algoritması ile Farklı Katlarda Ajan Tasarımı

Mini-Max algoritması ile farklı derinliklerde (2-kat, 3-kat, vb.) ileriye bakabilen ajanlar tasarladım. Derinlik arttıkça, ajanın yaptığı hamleler daha karmaşık ve doğru hale gelmektedir. Aşağıdaki kod, 2 derinlik seviyesinde Mini-Max algoritmasını kullanır:

```
best_move = minimax(game, 2, float('-inf'), float('inf'), True, players[current_player])
```

Bu kodda, 2 derinlik seviyesini belirtir. Bu seviyeyi artırarak ajanın daha ileri hamleleri analiz etmesini sağlayabilirsiniz.

Oyun Simülasyonu ve Skorlar

Ajanlar birbirlerine karşı 5 kez oynatıldığında, her oyun sonunda kazanan ve skor kaydedilir. Oyunların sonuçları aşağıdaki gibi hesaplanır:

```
if game.is_goal_state(1):
```

```
    winner = "White"
```

```
else:
```

```
    winner = "Black"
```

Ortalama Süre Hesaplamaları

Her oyun simülasyonu, tamamlanma süresini ölçmek için zamanlayıcı kullanılarak hesaplanmıştır. Aşağıdaki kod, her oyunun bitirme süresini hesaplar:

```
start_time = time.time()
```

```
# Move operation
```

```
end_time = time.time()
```

```
elapsed_time = end_time - start_time
```

Mini-Max algoritması ile Alfa-Beta budaması dahil edildiğinde, oyunların ortalama sürelerinde azalma gözlemlenmiştir çünkü Alfa-Beta budaması, gereksiz hesaplamaları atlayarak daha hızlı bir karar verme süreci sağlar.

Kod

```
from copy import deepcopy
```

```
import random
```

```
import time
```

```
class ChineseCheckers:
```

```
    def __init__(self):
```

```
        self.board = [[0] * 8 for _ in range(8)]
```

```
        for i in range(3):
```

```
            for j in range(3):
```

```
                self.board[i][j] = 1
```

```
        for i in range(5, 8):
```

```
            for j in range(5, 8):
```

```
                self.board[i][j] = 2
```

```
    def print_board(self):
```

```
        for row in self.board:
```

```
print(' '.join(str(cell) for cell in row))  
  
print()
```

```
def is_valid_move(self, x, y, nx, ny):
```

```
    if 0 <= nx < 8 and 0 <= ny < 8 and self.board[nx][ny] == 0:
```

```
        if abs(nx - x) <= 2 and abs(ny - y) <= 2:
```

```
            return True
```

```
    return False
```

```
def generate_moves(self, player):
```

```
    moves = []
```

```
    for x in range(8):
```

```
        for y in range(8):
```

```
            if self.board[x][y] == player:
```

```
                for dx in [-2, -1, 1, 2]:
```

```
                    for dy in [-2, -1, 1, 2]:
```

```
                        nx, ny = x + dx, y + dy
```

```
                        if self.is_valid_move(x, y, nx, ny):
```

```
                            moves.append(((x, y), (nx, ny)))
```

```
    return moves
```

```
def make_move(self, move):
```

```
    (x, y), (nx, ny) = move
```

```
self.board[nx][ny], self.board[x][y] = self.board[x][y], 0
```

```
def evaluate(self, player):
```

```
    score = 0
```

```
    for x in range(8):
```

```
        for y in range(8):
```

```
            if self.board[x][y] == player:
```

```
                score -= x + y if player == 1 else (7 - x) + (7 - y)
```

```
    return score
```

```
def is_goal_state(self, player):
```

```
    target = 1 if player == 2 else 2
```

```
    for x in range(5, 8) if target == 1 else range(0, 3):
```

```
        for y in range(5, 8) if target == 1 else range(0, 3):
```

```
            if self.board[x][y] != target:
```

```
                return False
```

```
    return True
```

```
def minimax(game, depth, alpha, beta, maximizing_player, player):
```

```
    if depth == 0 or game.is_goal_state(player):
```

```
        return game.evaluate(player), None
```

```
moves = game.generate_moves(player)
```

```
if not moves: # Eğer hiç hareket yoksa
```

```
    return game.evaluate(player), None
```

```
best_move = None
```

```
if maximizing_player:
```

```
    max_eval = float('-inf')
```

```
    for move in moves:
```

```
        new_game = deepcopy(game)
```

```
        new_game.make_move(move)
```

```
        eval, _ = minimax(new_game, depth - 1, alpha, beta, False, player)
```

```
        if eval > max_eval:
```

```
            max_eval = eval
```

```
            best_move = move
```

```
        alpha = max(alpha, eval)
```

```
        if beta <= alpha:
```

```
            break
```

```
    return max_eval, best_move
```

```
else:
```

```
    min_eval = float('inf')
```

```
    for move in moves:
```

```
        new_game = deepcopy(game)
```

```
        new_game.make_move(move)
```

```
        eval, _ = minimax(new_game, depth - 1, alpha, beta, True, player)
```

```
        if eval < min_eval:
```

```
        min_eval = eval
        best_move = move
    beta = min(beta, eval)
    if beta <= alpha:
        break
    return min_eval, best_move
```

```
def simulate_game():
    print("Game simulation starting...")
    game = ChineseCheckers()
    players = [1, 2]
    current_player = 0
    move_count = 0
    MAX_MOVES = 100

    print("Initial Board:")
    game.print_board()

    while not game.is_goal_state(players[current_player]) and move_count <
MAX_MOVES:
        print(f"Player {players[current_player]}'s turn:")
        _, move = minimax(game, 2, float('-inf'), float('inf'), True, players[current_player])
        if move:
            print(f"Player {players[current_player]} chooses move: {move}")
```

```
game.make_move(move)
```

```
move_count += 1
```

```
game.print_board()
```

```
else:
```

```
    print(f"Player {players[current_player]} has no valid moves! Exiting...")
```

```
    break
```

```
current_player = 1 - current_player
```

```
if move_count >= MAX_MOVES:
```

```
    print("Game terminated due to move limit!")
```

```
else:
```

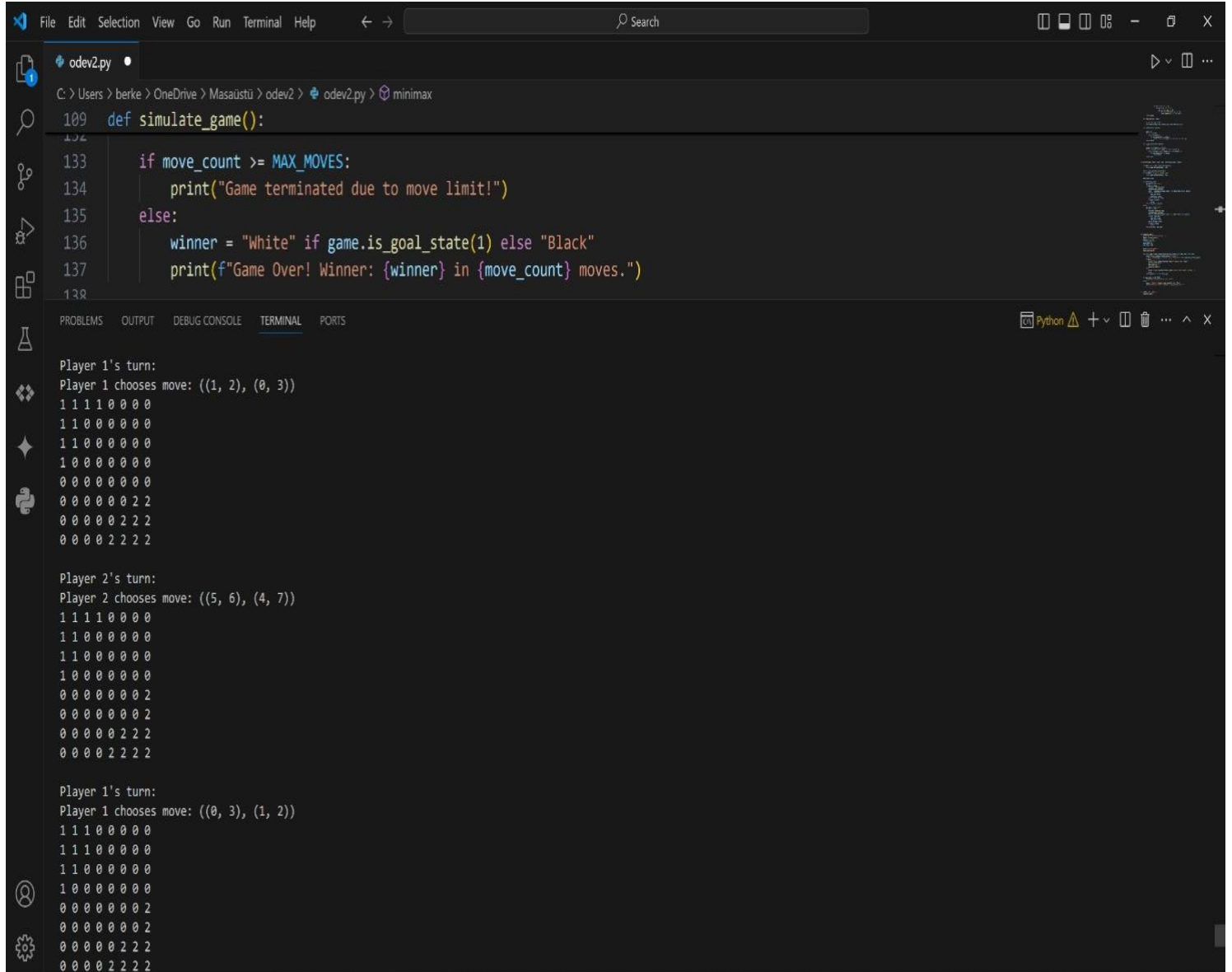
```
    winner = "White" if game.is_goal_state(1) else "Black"
```

```
    print(f"Game Over! Winner: {winner} in {move_count} moves.")
```

```
if __name__ == "__main__":
```

```
    simulate_game()
```


Farklı Tahta Durumlarında Yapılan Hamlelere Dair Ekran Çıktısı



The image shows a VS Code editor window with a Python file named `odev2.py` and a terminal window displaying the output of a game simulation.

Python Code (odev2.py):

```
109 def simulate_game():
133     if move_count >= MAX_MOVES:
134         print("Game terminated due to move limit!")
135     else:
136         winner = "White" if game.is_goal_state(1) else "Black"
137         print(f"Game Over! Winner: {winner} in {move_count} moves.")
138
```

Terminal Output:

```
Player 1's turn:
Player 1 chooses move: ((1, 2), (0, 3))
1 1 1 1 0 0 0 0
1 1 0 0 0 0 0 0
1 1 0 0 0 0 0 0
1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 2 2
0 0 0 0 0 2 2 2
0 0 0 0 2 2 2 2

Player 2's turn:
Player 2 chooses move: ((5, 6), (4, 7))
1 1 1 1 0 0 0 0
1 1 0 0 0 0 0 0
1 1 0 0 0 0 0 0
1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 2
0 0 0 0 0 0 0 2
0 0 0 0 0 2 2 2
0 0 0 0 2 2 2 2

Player 1's turn:
Player 1 chooses move: ((0, 3), (1, 2))
1 1 1 0 0 0 0 0
1 1 1 0 0 0 0 0
1 1 0 0 0 0 0 0
1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 2
0 0 0 0 0 0 0 2
0 0 0 0 0 2 2 2
0 0 0 0 2 2 2 2
```